

INTERNSHIP PROJECT REPORT

TASK MANAGEMENT SYSTEM

Submitted By

Name: Sandeep Seervi

USN: 24BTRCN034

Department: Computer Science and Engineering

Internship Organization

RD INFRO TECHNOLOGY

CERTIFICATE

This is to certify that **Mr. Sandeep Seervi**, bearing USN **24BTRCN034**, Department of Computer Science and Engineering, has successfully completed the internship project titled "**Task Management System**" at **RD INFRO TECHNOLOGY**.

The project work was carried out during the internship period and demonstrates practical implementation of full-stack web development concepts including frontend development, backend development, database integration, authentication, and cloud deployment.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **RD INFRO TECHNOLOGY** for providing me with the opportunity to undertake this internship and work on the Task Management System project.

I would also like to thank my mentors, faculty members, and everyone who supported and guided me throughout the development of this project. This internship helped me gain valuable practical knowledge in web development, database management, authentication systems, API development, and cloud deployment.

The experience gained during this internship has enhanced my technical skills and provided me with exposure to real-world software development practices.

Sandeep

USN: 24BTRCN034

Seervi

ABSTRACT

The Task Management System is a full-stack web application developed to help users organize and manage their daily tasks efficiently. The application provides functionalities such as user registration, secure login, task creation, task editing, task deletion, and logout.

The frontend of the application is developed using HTML, CSS, and JavaScript, while the backend is developed using Node.js and Express.js. MongoDB Atlas is used as the cloud database for storing user and task information. Password security is implemented using bcrypt password hashing, and JSON Web Tokens (JWT) are used for authentication.

The project demonstrates practical implementation of frontend development, backend API development, database connectivity, authentication mechanisms, CRUD operations, version control, and cloud deployment.

Table of Contents

Sl.No.	Title	Page No.
1	Introduction	04
2	Objectives	04
3	Technologies used	04-05
4	System Architecture	05-06
5	Project modules	06
6	Database Design	06-07
7	Implementation	07-08
8	Results and Screenshots	08-14
9	Advantages	14
10	Future Enhancements	14-15
11	Conclusion	15
12	References	15
13	Project Links	16

CHAPTER 1: INTRODUCTION

Task management is an essential aspect of personal and professional productivity. Managing tasks efficiently helps users organize their work, track progress, and improve overall performance.

The Task Management System is designed to provide users with a simple and efficient platform for managing daily tasks. The application allows users to create accounts, log in securely, and perform task management operations such as creating, updating, and deleting tasks.

The project aims to provide hands-on experience in full-stack web development by integrating frontend technologies, backend APIs, database management systems, authentication mechanisms, and cloud deployment platforms.

CHAPTER 2: OBJECTIVES

The main objectives of this project are:

- To develop a user-friendly task management application.
 - To implement secure user authentication.
 - To perform CRUD (Create, Read, Update, Delete) operations on tasks.
 - To integrate MongoDB Atlas for cloud-based data storage.
 - To implement password encryption and authentication mechanisms.
 - To gain practical experience in frontend and backend development.
 - To deploy the application using cloud hosting platforms.
 - To understand real-world software development practices.
-

CHAPTER 3: TECHNOLOGIES USED

The Task Management System was developed using a combination of frontend, backend, database, authentication, and deployment technologies.

HTML (HyperText Markup Language) was used to create the structure and layout of the web pages including the Home Page, Registration Page, Login Page, and Dashboard.

CSS (Cascading Style Sheets) was used to design and style the user interface. It helped create a visually appealing and responsive layout for users.

JavaScript was used to implement client-side functionality, validate user inputs, handle events, and communicate with backend APIs.

Node.js was used as the server-side runtime environment. It enabled JavaScript execution on the server and facilitated backend development.

Express.js was used as the web application framework. It simplified routing, middleware handling, and API development.

MongoDB Atlas was used as the cloud database to store user information and task data securely.

bcryptjs was used to encrypt user passwords before storing them in the database.

JSON Web Token (JWT) was used to authenticate users and provide secure access to application resources.

GitHub was used for source code management and version control throughout the development process.

Railway was used to deploy and host the backend application.

Netlify was used to deploy and host the frontend application.

CHAPTER 4: SYSTEM ARCHITECTURE

The system follows a client-server architecture.

The user interacts with the frontend interface developed using HTML, CSS, and JavaScript. User requests are sent to the backend through REST APIs developed using Express.js and Node.js.

The backend processes the requests and interacts with MongoDB Atlas to store or retrieve data. Authentication is handled using JWT, while user passwords are securely encrypted using bcrypt before being stored in the database.

Workflow

1. User registers using the registration page.
2. Password is encrypted using bcrypt.
3. User information is stored in MongoDB Atlas.
4. User logs in using email and password.
5. JWT token is generated after successful authentication.
6. User accesses the dashboard.

7. User performs task operations.
8. Task information is stored and retrieved from MongoDB Atlas.
9. Results are displayed on the dashboard.

CHAPTER 5: PROJECT MODULES

User Registration Module

This module allows new users to create an account by providing their name, email address, and password. Passwords are encrypted before storage.

User Login Module

This module authenticates registered users using email and password credentials. JWT tokens are generated upon successful login.

Dashboard Module

The dashboard provides users with an interface to view and manage tasks.

Task Creation Module

This module allows users to create new tasks by entering task titles and descriptions.

Task Management Module

This module allows users to edit, update, and delete existing tasks.

Logout Module

This module allows users to securely end their session by removing user data from local storage.

CHAPTER 6: DATABASE DESIGN

MongoDB Atlas was used as the cloud database solution for this project.

Users Collection

The Users collection stores information related to registered users. Each user document contains a unique identifier, name, email address, and encrypted password.

The password is stored in hashed format using bcrypt, ensuring better security and protection of sensitive user information.

Tasks Collection

The Tasks collection stores information related to tasks created within the application.

Each task document contains:

- Task Title
- Task Description
- Completion Status
- Creation Timestamp
- Update Timestamp

This collection supports all CRUD operations performed by users through the dashboard.

CHAPTER 7: IMPLEMENTATION

The implementation phase involved developing both frontend and backend components of the application.

Frontend Implementation

The frontend was developed using HTML, CSS, and JavaScript.

The main pages include:

- Home Page
- Registration Page
- Login Page
- Dashboard Page

JavaScript was used to handle user interactions and communicate with backend APIs.

Backend Implementation

The backend was developed using Node.js and Express.js.

Major backend files include:

- server.js
- authcontroller.js
- taskcontroller.js
- auth.js
- task.js

The backend handles authentication, task management, API requests, and database operations.

Database Integration

MongoDB Atlas was integrated with the backend using Mongoose. User and task information are stored and retrieved through database queries.

Deployment

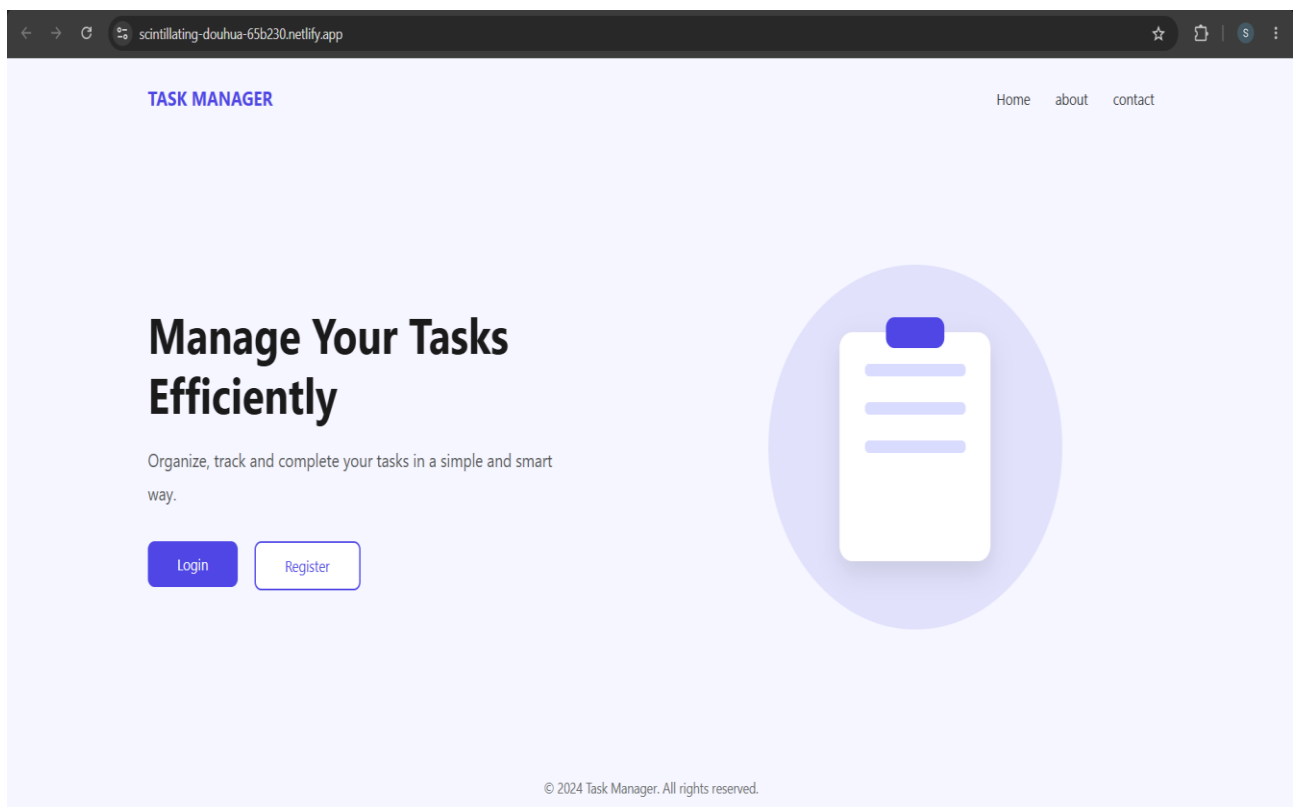
The backend application was deployed on Railway.

The frontend application was deployed on Netlify.

The source code was maintained using GitHub.

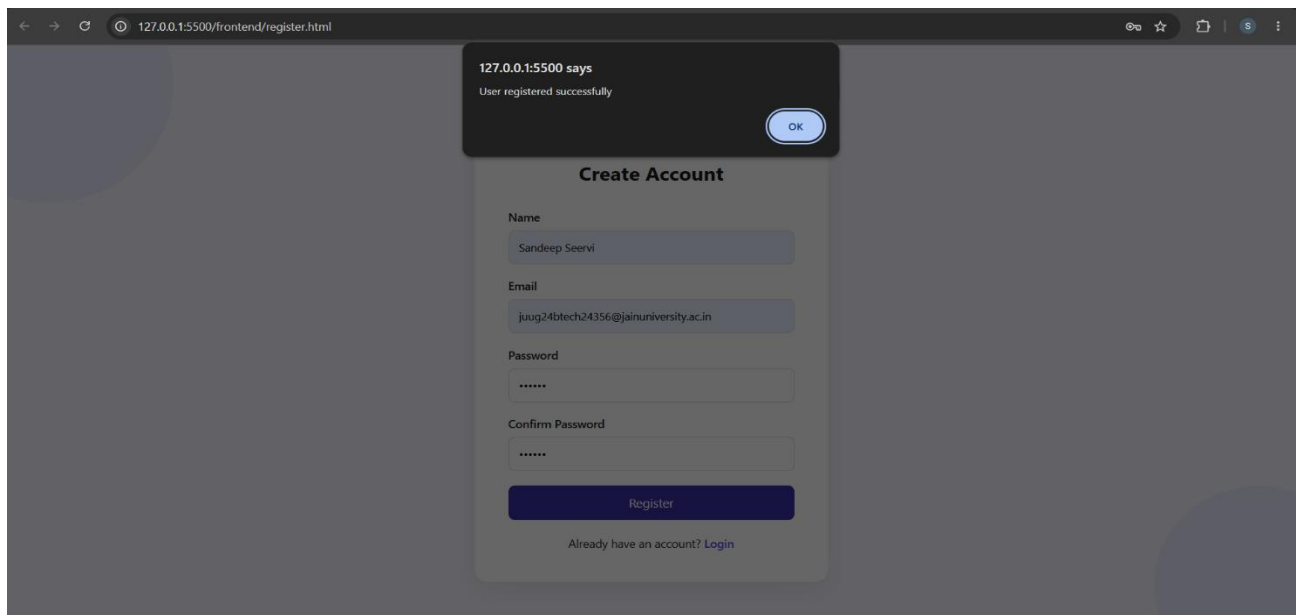
CHAPTER 8: RESULTS AND SCREENSHOTS

Figure 8.1: Home Page



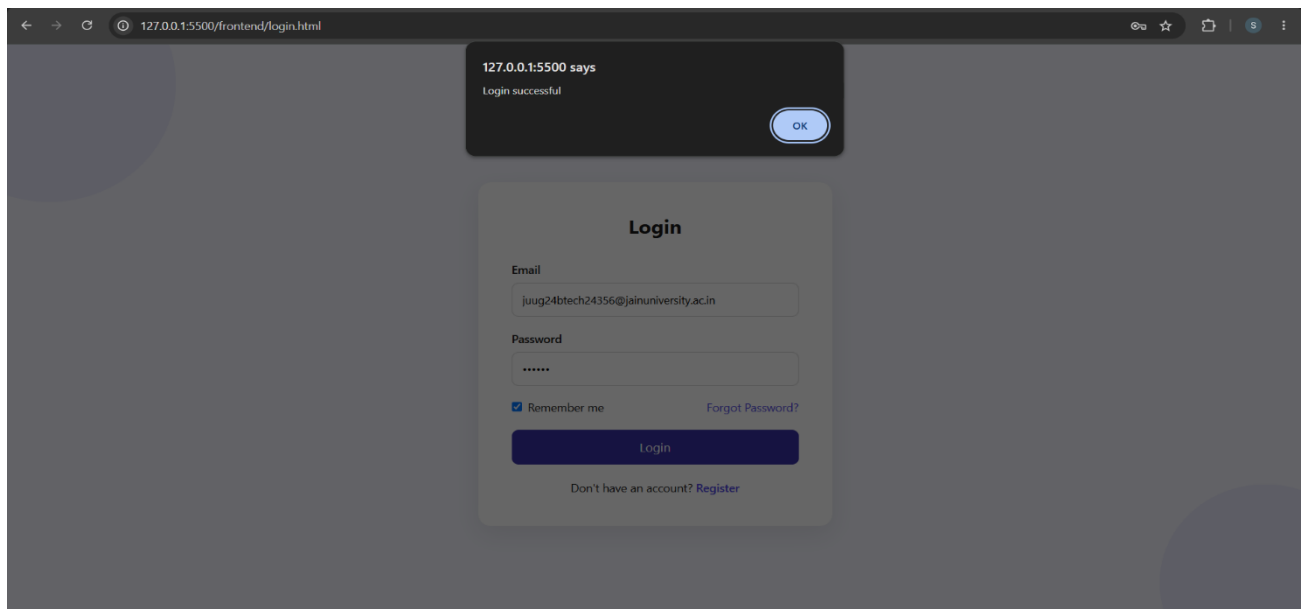
Description: The home page serves as the entry point of the application and provides navigation options for registration and login.

Figure 8.2: Registration Page



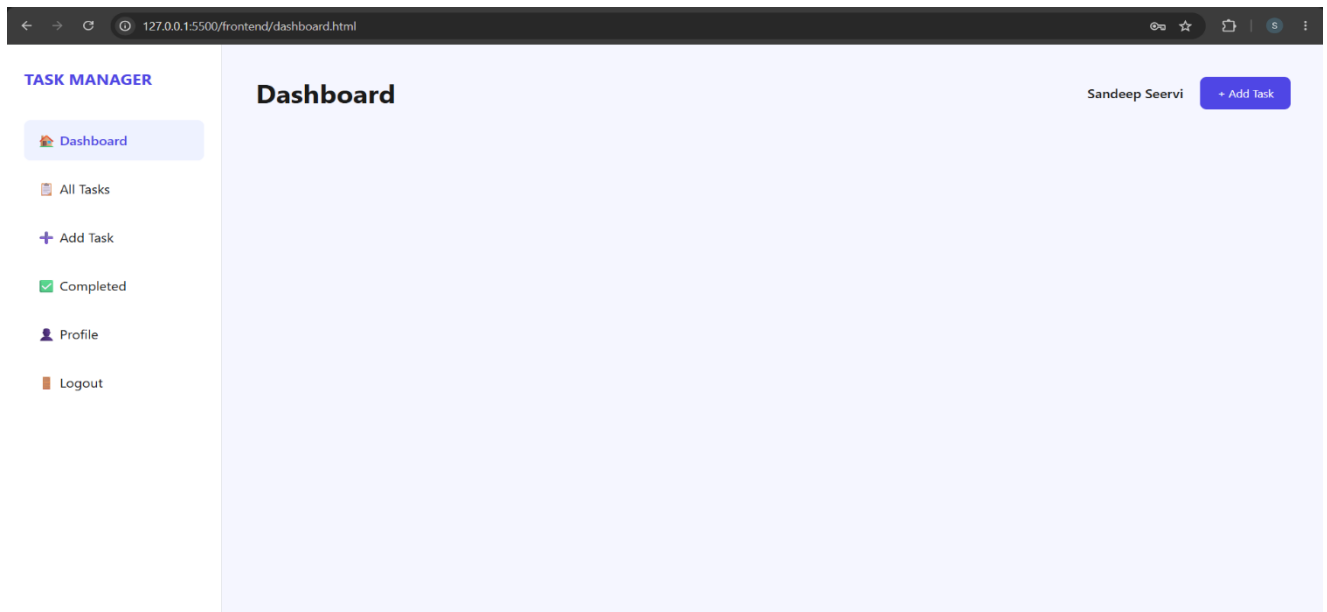
Description: Users can create a new account by providing their personal details.

Figure 8.3: Login Page



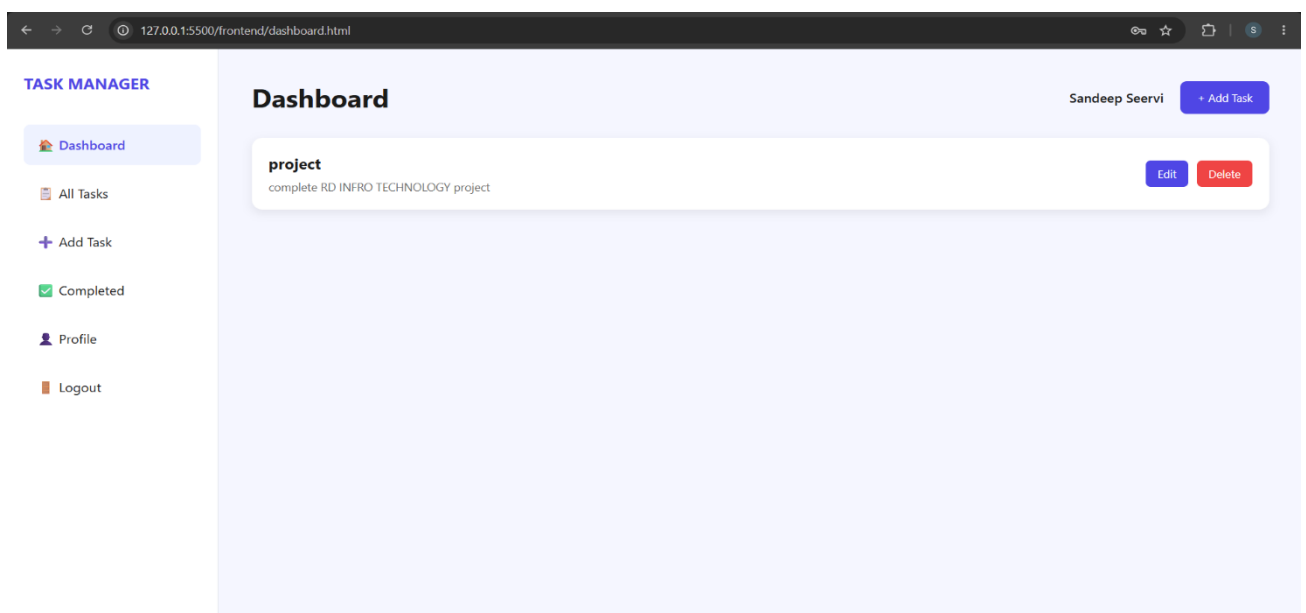
Description: Registered users can securely access the application using their credentials.

Figure 8.4: Dashboard Page



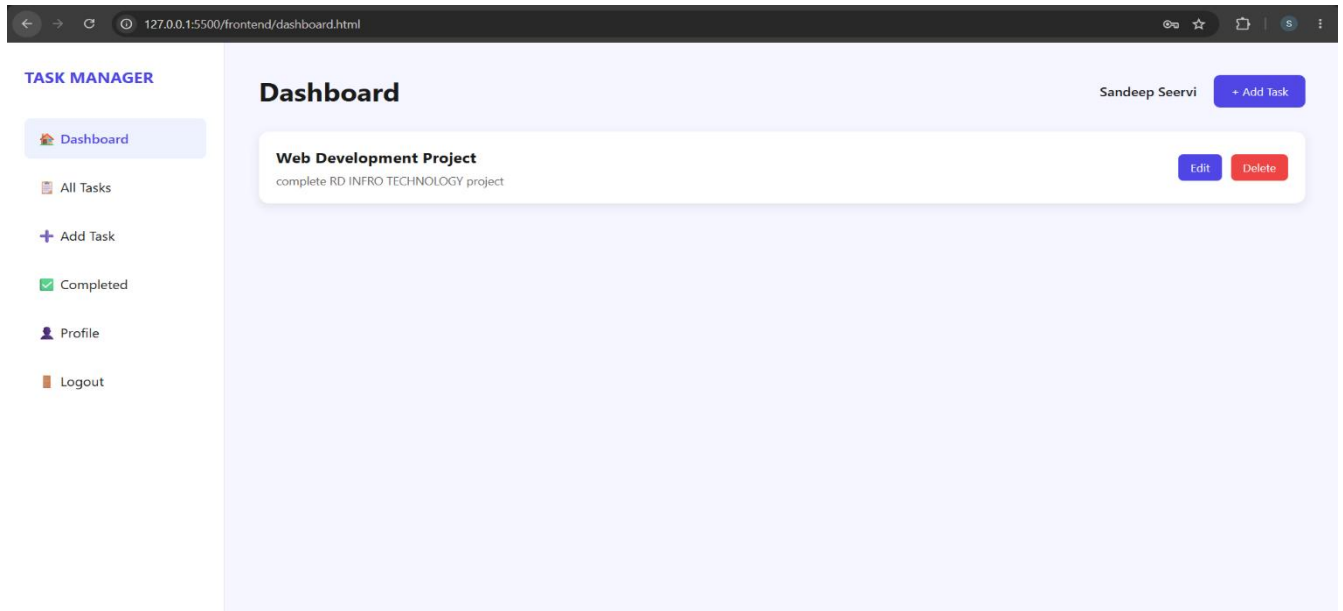
Description: The dashboard displays tasks and provides task management functionalities.

Figure 8.5: Add Task Functionality



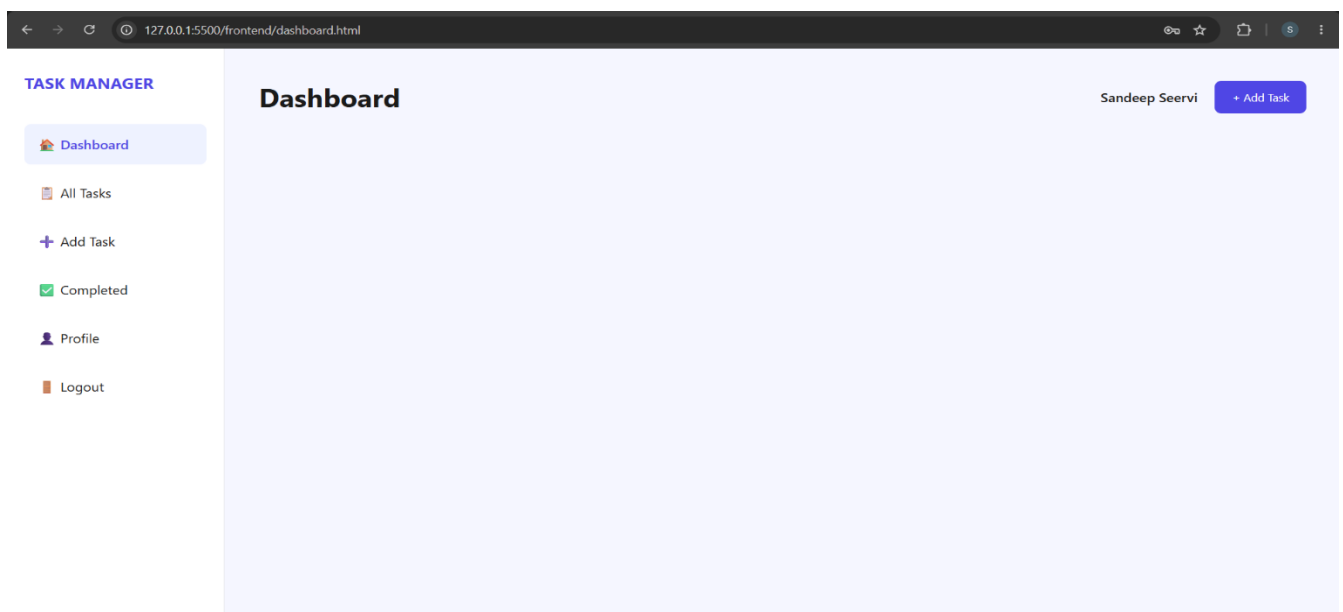
Description: Users can create and store new tasks in the system.

Figure 8.6: Edit Task Functionality



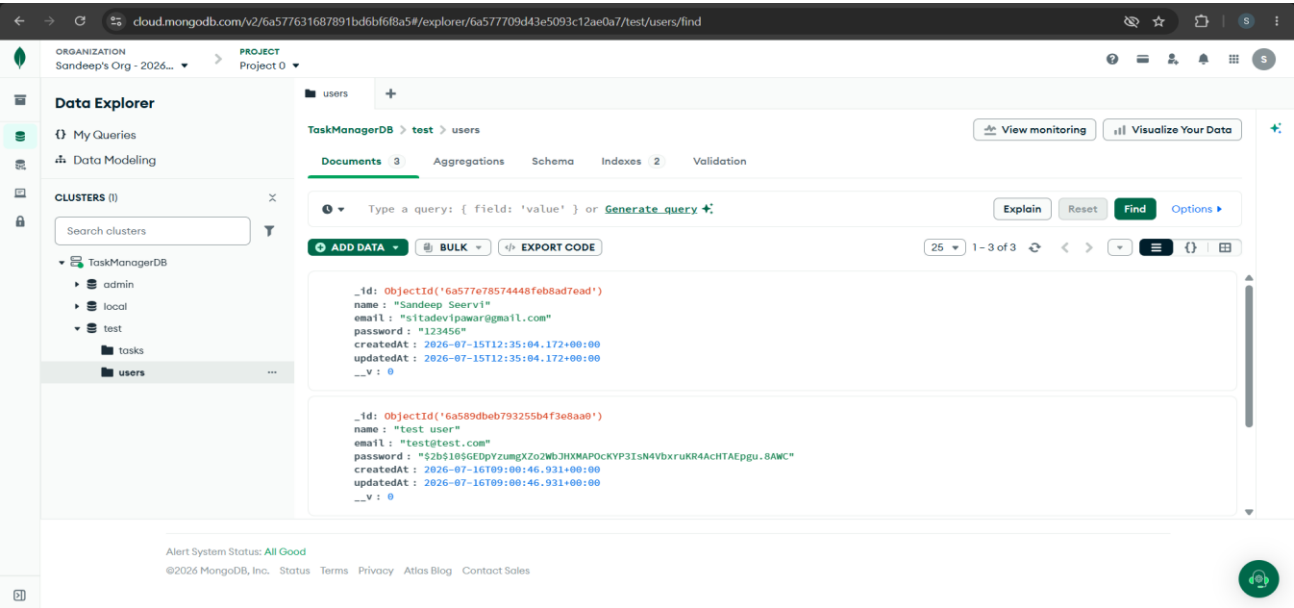
Description: Existing tasks can be updated or modified.

Figure 8.7: Delete Task Functionality



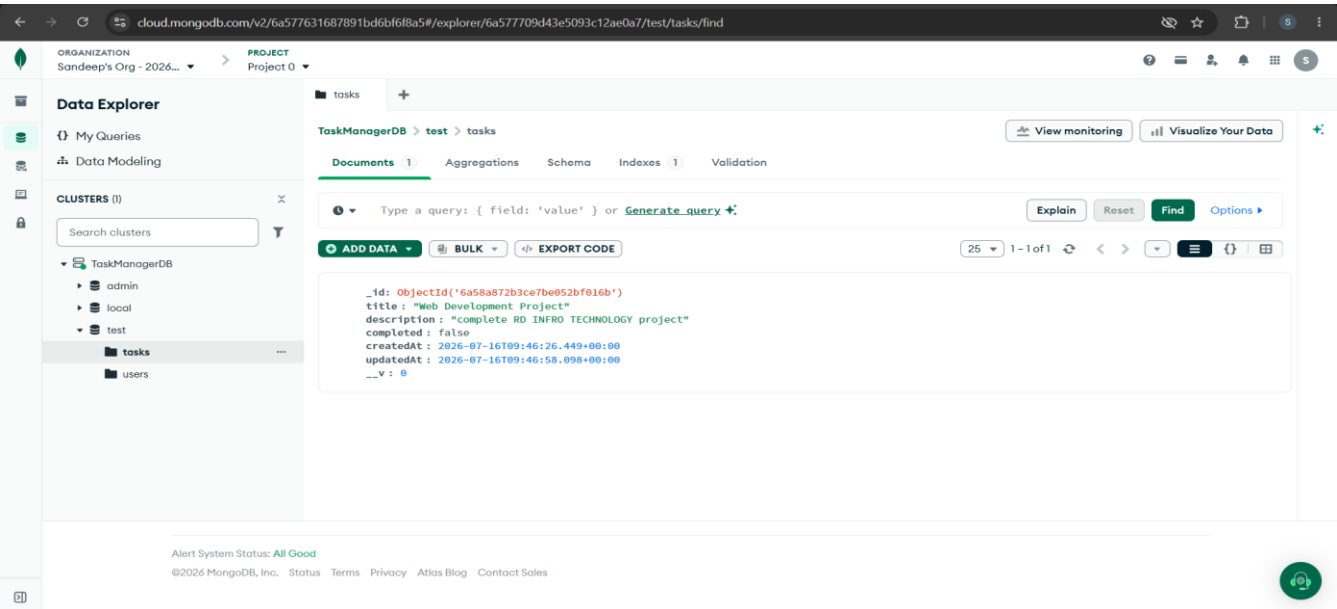
Description: Users can remove unnecessary tasks from the system.

Figure 8.8: MongoDB Users Collection



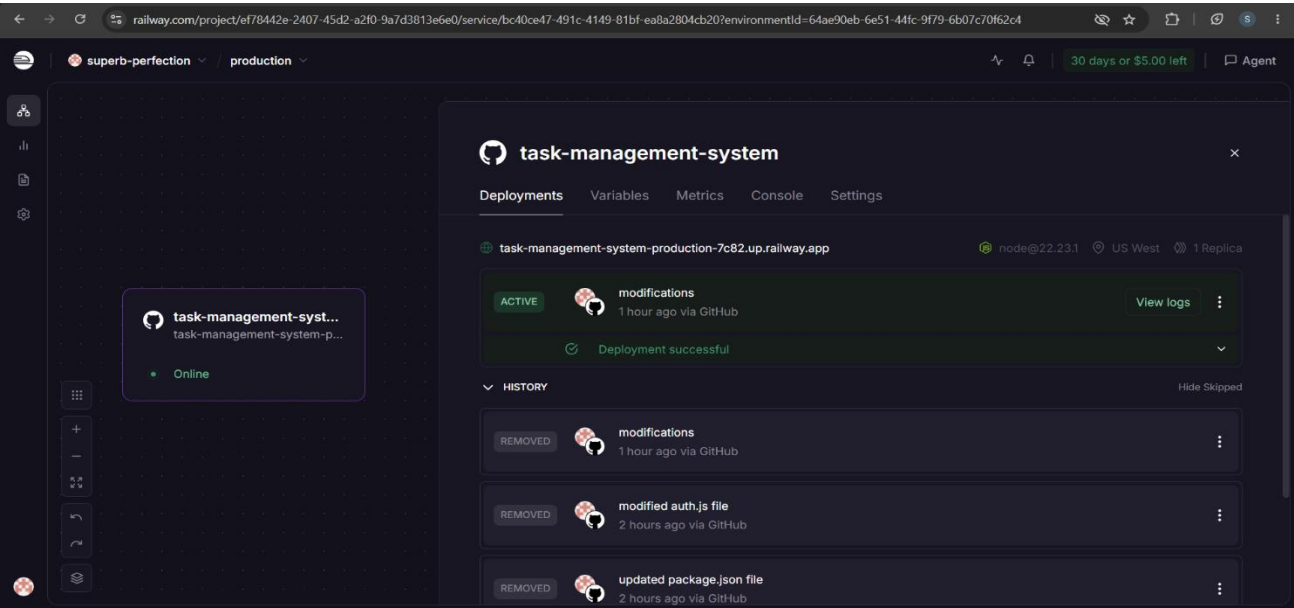
Description: Shows user information stored in MongoDB Atlas.

Figure 8.9: MongoDB Tasks Collection



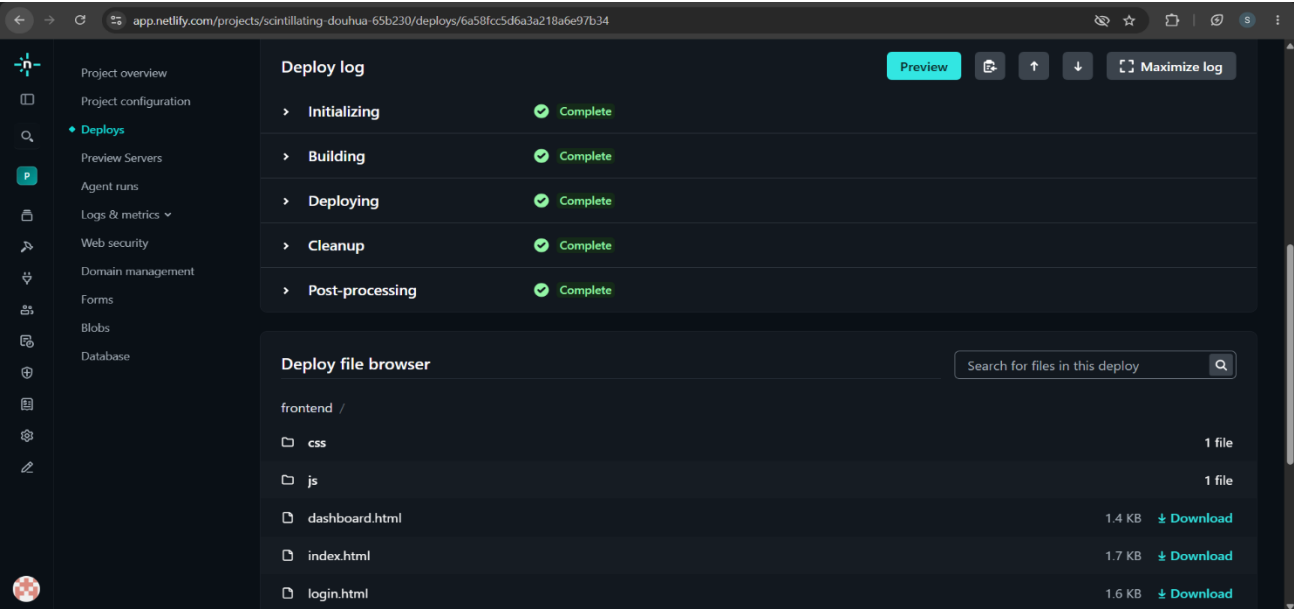
Description: Shows task information stored in MongoDB Atlas.

Figure 8.10: Railway Deployment



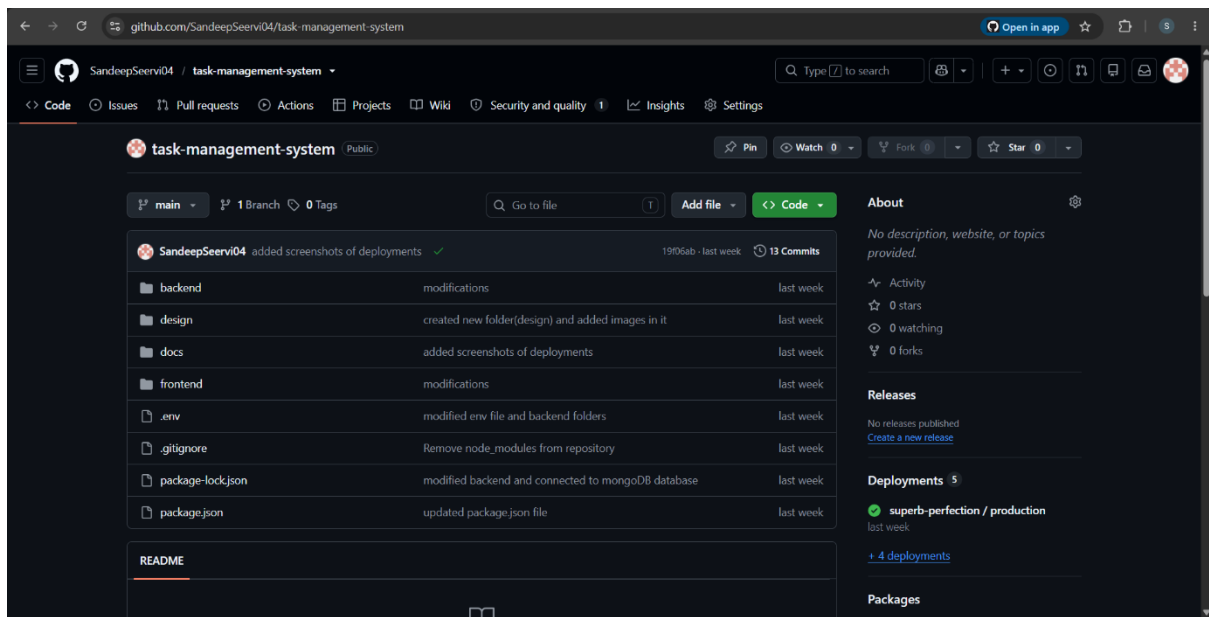
Description: Displays successful backend deployment on Railway.

Figure 8.11: Netlify Deployment



Description: Displays successful frontend deployment on Netlify.

Figure 8.12: GitHub Repository



Description: Shows project source code hosted on GitHub.

CHAPTER 9: ADVANTAGES

The advantages of the Task Management System include:

- Simple and user-friendly interface.
- Secure user authentication.
- Password encryption for enhanced security.
- Cloud-based database storage.
- Efficient task management.
- Easy accessibility through cloud deployment.
- Real-time interaction with backend APIs.
- Practical implementation of full-stack development concepts.

CHAPTER 10: FUTURE ENHANCEMENTS

The following features can be added in future versions:

- User-specific task filtering.
- Task priority levels.

- Due date reminders.
 - Task completion tracking.
 - Email notifications.
 - Mobile application version.
 - Advanced dashboard analytics.
 - Profile management functionality.
-

CHAPTER 11: CONCLUSION

The Task Management System was successfully designed, developed, tested, and deployed using modern web development technologies.

The project provided practical exposure to frontend development, backend development, database integration, authentication systems, cloud deployment, and version control. The application successfully implements user authentication and task management functionalities while maintaining data security through password encryption and JWT authentication.

Overall, the project achieved its objectives and served as an excellent learning experience in full-stack web development.

CHAPTER 12: REFERENCES

1. Node.js Official Documentation
2. Express.js Official Documentation
3. MongoDB Atlas Documentation
4. JWT Documentation
5. bcryptjs Documentation
6. GitHub Documentation
7. Railway Documentation
8. Netlify Documentation
9. MDN Web Docs
10. JavaScript Documentation

CHAPTER 13: PROJECT LINKS

1. Github repository link: <https://github.com/SandeepSeervi04/task-management-system>
2. Netlify frontend deployment link: <https://scintillating-douhua-65b230.netlify.app/>
3. Backend Deployment:
<https://task-management-system-production-7c82.up.railway.app>